OpenClaw Bible

🐾 The OpenClaw Bible

A Complete Guide to Self-Hosted AI Agents

By Dr Rushindra Sinha (Rushi) — battle-tested with agents Ares & Sam *Last updated: February 2026*

📄 Table of Contents

1. What is OpenClaw?
2. Prerequisites
3. Installation
4. Directory Structure
5. The Soul Stack (Agent Config Files)
6. Channel Setup
7. Token & API Key Management
8. Usage Monitoring & Cost Control
9. Security & Safety
10. Skills System
11. Cron Jobs & Heartbeats
12. Memory System Deep Dive
13. Multi-Agent Setup
14. Google Drive Integration
15. Media Production Setup
16. Large File Handling
17. Common Issues & Fixes
18. Prompt Engineering Bible
19. Cost Estimates
20. Quick Reference Card

---

1. 🐾 What is OpenClaw?

OpenClaw is a **self-hosted AI agent gateway**. It is not a cloud service. You run it on your own hardware — a Mac mini, MacBook, VPS, or even a Raspberry Pi — and it connects messaging platforms (WhatsApp, Telegram, Discord, iMessage, Signal) to AI models (Claude, GPT, etc.).

The result: an AI agent that lives in your chat apps, has access to your machine, and can execute real tasks — file management, web browsing, media production, API calls, cron jobs, and more.

Key facts: - 🏠 **Self-hosted** — your data stays on your machine - 🔌 **Multi-channel** — one agent, many chat platforms simultaneously - 🤖 **Multi-model** — Claude (Anthropic), GPT (OpenAI), Gemini (Google), and others - 🛠️ **Full machine access** — shell commands, file system, browser, camera, network - ⚠️ **Intentionally hard to set up** — the founder designed it this way to prevent security incidents from casual users. If you can't install it, you probably shouldn't be running it.

> 💡 Think of it as SSH + AI + messaging glue. Your agent is a remote terminal operator that speaks human.

---

2. 📋 Prerequisites

Requirement	Details
Node.js 22+	`node --version` — must be v22 or higher
OS	macOS (primary, best supported), Linux, Windows (PowerShell)
API Key	Anthropic (Claude) recommended. You need at least one LLM key.
Terminal comfort	`cd` , `ls` , `cat` , `nano` / `vim` — you'll be editing config files
Homebrew (macOS)	`/bin/bash -c "$(curl -fsSL https://`

Requirement	Details
	`raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"`
Git	Optional but useful for version-controlling your soul files

✅ **Recommended first-time model:** Claude Sonnet 4.5 or 4.6 — best cost/quality ratio for daily use.

---

3. 🔧 Installation

```
# 1. Install OpenClaw
curl -fsSL https://openclaw.ai/install.sh | bash

# 2. Run onboarding (sets up daemon, creates config)
openclaw onboard --install-daemon

# 3. Check gateway status
openclaw gateway status

# 4. Open the dashboard (web UI)
openclaw dashboard
```

What each step does: - `install.sh` — downloads the OpenClaw binary, adds it to PATH - `onboard --install-daemon` — creates `~/.openclaw/`, generates `openclaw.json`, installs the background daemon (launchd on macOS, systemd on Linux) - `gateway status` — confirms the daemon is running and healthy - `dashboard` — opens a local web UI for configuration and monitoring

Common installation issues:

Problem	Fix
`node: command not found`	Install Node 22+: `brew install node@22`
Permission denied on install	Run with `sudo` or fix `/usr/local` ownership

Problem	Fix
Gateway won't start	Check `~/.openclaw/logs/` for errors. Usually a port conflict or missing config.
`openclaw: command not found` after install	Restart your terminal or `source ~/.zshrc`

---

4. 📂 Directory Structure

```
~/.openclaw/                    # Runtime data (managed by OpenClaw)
├── openclaw.json               # ⭐ Main config — API keys, channels,
env vars
├── media/                      # Media files for sending/receiving
├── logs/                       # Gateway and agent logs
└── skills/                     # Installed skills from ClawHub

~/clawd/                        # Default agent workspace
├── SOUL.md                     # ⭐ Agent personality & capabilities
├── AGENTS.md                   # Operational protocols & workflows
├── MEMORY.md                   # Long-term persistent memory
├── HEARTBEAT.md                # Periodic/cron task definitions
├── TOOLS.md                    # Environment-specific notes
├── USER.md                     # Info about the human operator
├── IDENTITY.md                 # Agent name, emoji, vibe
├── BOOTSTRAP.md                # First-run guide (delete after setup)
└── memory/                     # Daily session logs
    ├── 2026-02-19.md
    ├── 2026-02-20.md
    └── 2026-02-21.md
```

For multi-agent setups, each agent gets its own workspace:

```
~/ares/                         # Ares — ops/strategy agent
~/sam/                          # Sam — content/media agent
```

---

5. 🧠 The Soul Stack

> 💡 **"The secret is not in the skills.md file — it's in the soul.md file."** Skills define HOW to use a tool. The Soul defines WHO the agent is, WHEN it acts, and WHY.

This is the most important section. These files define your agent's entire being.

SOUL.md — The Core Identity

The single most important file. Defines personality, tone, capabilities, boundaries, and decision-making framework.

```
# SOUL.md — Ares

You are Ares, a strategic operations agent for Dr Rushindra Sinha.
You handle: business ops, scheduling, research, trading signals,
        memory management.
You do NOT handle: content production (that's Sam's job).

## Tone
Direct, efficient, no fluff. Military precision with dry humor.

## Boundaries
- Never send DMs to anyone without explicit permission
- Never execute financial transactions without confirmation
- Always confirm before deleting files
```

AGENTS.md — Operational Protocols

Workflows, pipelines, standard operating procedures. Think of it as the agent's employee handbook.

MEMORY.md — Long-term Memory

Curated facts, decisions, preferences, protocols. This persists across sessions and context compactions. The agent refers to this as ground truth.

memory/YYYY-MM-DD.md — Daily Logs

Auto-generated session logs. Captures what happened each day — tasks completed, decisions made, blockers hit.

HEARTBEAT.md — Periodic Tasks

Defines cron-like jobs: news checks, system health, memory flushes, scheduled posts.

USER.md — About the Human

Who you are, your preferences, communication style, time zone, key contacts.

IDENTITY.md — Agent Identity

Name, creature type, emoji, personality vibe. Cosmetic but important for multi-agent setups.

TOOLS.md — Environment Notes

Your specific setup: camera names, SSH hosts, preferred TTS voices, device nicknames. Not skills — just your local cheat sheet.

BOOTSTRAP.md — First Run

One-time setup instructions. Delete after initial configuration is complete.

---

6. 📡 Channel Setup

WhatsApp (Most Popular)

```
# WhatsApp pairs via QR code through the gateway
openclaw gateway status    # Ensure gateway is running
# Follow QR code prompt in terminal or dashboard
# Scan with WhatsApp > Linked Devices > Link a Device
```

⚠️ **CRITICAL: WhatsApp has a 16MB media file limit.** For HD videos, large audio files, or any file >16MB — upload to Google Drive and share the link. Never try to send large files directly.

Telegram

1. Message @BotFather on Telegram
2. `/newbot` → follow prompts → get your bot token
3. Add token to `~/.openclaw/openclaw.json` under the Telegram channel config
4. Start the bot, send it a message

Discord

1. Create a bot at Discord Developer Portal
2. Get bot token, enable Message Content Intent
3. Add to `openclaw.json`, invite bot to your server with appropriate permissions

iMessage (macOS only)

Requires BlueBubbles running on macOS. Complex setup — only recommended if you're already in the Apple ecosystem.

Signal

Supported but less commonly used. Requires Signal CLI setup.

---

7. 🔑 Token & API Key Management

All keys live in `~/.openclaw/openclaw.json` under the `env` section:

```
{
  "env": {
    "ANTHROPIC_API_KEY": "sk-ant-...",
    "OPENAI_API_KEY": "sk-...",
    "ELEVENLABS_API_KEY": "...",
    "PEXELS_API_KEY": "...",
    "GEMINI_API_KEY": "..."
  }
}
```

These are injected at runtime and available to all agents as environment variables.

🔐 1Password Integration (Recommended)

```
# Install 1Password CLI
brew install --cask 1password-cli

# Sign in
op signin

# Reference secrets in openclaw.json using op:// URIs
# OpenClaw resolves them at runtime
```

> ❌ **NEVER hardcode API keys in scripts.** Use env vars or `op` CLI references.

Model Options & Costs

Model	Best For	Relative Cost
Claude Sonnet 4.5/4.6	Daily driver — best cost/quality	||* * *ClaudeOpus*4.6 * *|*Complexdocs*, *deepreasoning*| $$ \| \| **Claude Haiku 3** \| Quick lookups, simple tasks \| $ \| \| **GPT-5.1 / 5.2** \| Alternative, good for some tasks \| $$ -$$$

---

8. 📊 Usage Monitoring & Cost Control

```
/status          # In any chat — shows current model, usage, cost
/reasoning       # Toggle reasoning display
/verbose         # Toggle verbose mode
```

Model aliases in chat: `sonnet`, `haiku`, `Opus46`, `gpt`, `GPT` — use these to switch models mid-conversation.

💡 **Cost-saving strategy:** - **Haiku** for simple Q&A, lookups, quick translations - **Sonnet** for standard work — 90% of your usage - **Opus** only for complex writing, deep analysis, multi-step reasoning

Monitor spend at console.anthropic.com → Usage dashboard.

---

9. 🔒 Security & Safety

> ⚠️ **This is the most important chapter. Read it twice.**

What the Agent Can Do

The agent runs with YOUR user permissions. It can: - Execute **any shell command** on your machine - Read/write **any file** you have access to - Access your **browser**, **camera** (via Peekaboo skill), **microphone** - Send messages **as you** on WhatsApp, Telegram, Discord - Browse the internet, make API calls, download files - Install packages, modify system configs

A misconfigured agent can delete files, expose credentials, send unsolicited messages, or worse.

Safety Layers

✅ **SOUL.md boundaries** — explicitly define what the agent CAN and CANNOT do ✅ `ask: on-miss` **exec mode** — agent prompts before running unknown commands ✅ **AGENTS.md protocols** — define exact allowed operations ✅ **ClawVault (1Password)** — scoped secrets, not raw passwords ❌ **Never** give the agent banking passwords outside of ClawVault ❌ **Never** let the agent add itself to groups without permission

Kill Switches (Memorize These)

```
# 🔴 Nuclear: Stop everything
openclaw gateway stop

# 🟡 Soft kill: Remove soul (agent loses context on restart)
mv ~/clawd/SOUL.md ~/clawd/SOUL.md.disabled

# 🟡 API kill: Remove key (agent can't call AI)
# Edit ~/.openclaw/openclaw.json → remove ANTHROPIC_API_KEY
```

```
# 🔴 Process kill
kill <PID>     # Kill specific agent process
```

🧅 The Sandbox Approach (Rushi's Method)

Start locked down. Peel layers slowly.

1. **Week 1:** Chatbot only — no file access, no shell, no channels
2. **Week 2:** Add file read access to workspace only
3. **Week 3:** Add shell execution with `ask: on-miss`
4. **Week 4:** Add one channel (WhatsApp personal chat)
5. **Week 5+:** Add internet, APIs, additional channels, cron jobs

> 💡 Verify each new capability in a test session before production use. Trust is earned.

Group Chat Safety

- Agent responds **only when @mentioned** in groups
- No DMs to group participants without explicit permission
- Establish command priority: who can give orders, in what order

---

10. 🧰 Skills System

Skills are pre-built tool integrations available via ClawHub.

```
clawhub search whisper        # Search for skills
clawhub install whisper       # Install a skill
```

Key skills on Rushi's setup:

Skill	Purpose
`gog`	Google API integration (Drive, Sheets, Docs)
`whisper`	Audio transcription (OpenAI Whisper)
`coding-agent`	Code generation and debugging

Skill	Purpose
`peekaboo`	Camera access and snapshots
`nano-banana-pro`	Image generation via Gemini (up to 4K)
`wacli`	WhatsApp CLI tools
`himalaya`	Email access
`sag`	Sub-agent spawning
`gifgrep`	GIF search and reactions

Writing Custom Skills

Create a `SKILL.md` in your skill directory defining the tool's interface, plus any scripts needed. See clawhub.com for examples.

> 💡 **Skills vs Soul:** Skills = HOW to use a tool. Soul = WHETHER and WHY to use it. A well-written SOUL.md with no skills is more useful than 50 skills with a bad soul.

---

11. ⏰ Cron Jobs & Heartbeats

`HEARTBEAT.md` defines periodic tasks the agent runs automatically.

Common heartbeat categories: - 📰 **News monitoring** — check RSS feeds, send summaries - 🎬 **Content pipeline** — auto-draft and queue content on schedule - 💻 **System health** — check disk space, gateway status, API quotas - 🧠 **Memory flush** — auto-save context before compaction - 📱 **Channel monitoring** — watch for mentions, DMs, important messages - 📈 **Trading signals** (advanced) — crypto/stock alerts (Ares use case) - 📲 **Social media** — scheduled posts, engagement monitoring

When there's nothing to report, the agent responds with `HEARTBEAT_OK`.

---

12. 🧠 Memory System Deep Dive

OpenClaw uses a **two-layer memory system:**

Layer	File	Purpose	Persistence
Long-term	`MEMORY.md`	Curated facts, protocols, decisions	Permanent until edited
Daily	`memory/YYYY-MM-DD.md`	Session logs, task results	Permanent, one per day

Key Memory Tools

- `memory_search` — search across all memory files
- `memory_get` — retrieve specific memory entries

When to Write Memory

Write to memory when: decisions are made, protocols change, lessons are learned, blockers are discovered, preferences are stated.

Best Practices

✅ End every productive session with: **"Distill this session to memory"** ✅ Review MEMORY.md monthly — prune outdated entries ✅ Memory auto-flushes before context compaction (when conversation gets long) ❌ Don't rely on the agent "remembering" — if it's not written down, it doesn't exist

---

13. 🤝 Multi-Agent Setup

Rushi runs two agents on one Mac mini:

Agent	Workspace	Role
Ares	`~/ares/`	Ops, strategy, scheduling, research, trading
Sam	`~/sam/`	Content production, media, YouTube pipeline

How They Coordinate

- `sessions_send` — send messages between agents
- `sessions_list` — see active agent sessions
- `subagents` — spawn child agents for specific tasks

Shared Resources

- Google Drive as common storage
- Shared env vars in `openclaw.json`
- Cross-references in each agent's SOUL.md ("For content tasks, delegate to Sam")

---

14. 📁 Google Drive Integration

```python
# Use Google API Python client directly
from google.oauth2.credentials import Credentials
from googleapiclient.discovery import build

creds = Credentials.from_authorized_user_file(
    '~/.config/ares/google/token.json',
    scopes=['https://www.googleapis.com/auth/drive']
)
service = build('drive', 'v3', credentials=creds)
```

File Delivery Protocol (Standard)

1. Save file locally to `~/Desktop/GEWR/<subfolder>/`
2. Upload to Drive GEWR folder
3. Share Drive link in chat

Finding Folder IDs

Open the folder in Google Drive web → the URL contains the folder ID:
`https://drive.google.com/drive/folders/<FOLDER_ID>`

> 💡 Keep separate tokens: `token.json` for Drive/Sheets, `youtube_token.json` for YouTube uploads. Different scopes.

---

15. 🎬 Media Production Setup (Sam's Stack)

API Services

Service	Key	Purpose
ElevenLabs	`ELEVENLABS_API_KEY`	Text-to-speech, voice notes
Pexels	`PEXELS_API_KEY`	Stock footage and images
Gemini	`GEMINI_API_KEY`	Image generation (nano-banana-pro, up to 4K)

ffmpeg on macOS

```
brew install ffmpeg
ffmpeg -buildconf    # Check compiled features
```

⚠️ **LESSON: Homebrew ffmpeg 8.0.1 does NOT include drawtext** (libfreetype not compiled in). For text overlays, use: - CapCut or Premiere for manual editing - Python Pillow library for programmatic text-on-image - Custom ffmpeg build with `--enable-libfreetype`

Whisper (Audio Transcription)

```
pip install openai-whisper
```

Model	Speed (8-min video, CPU)	Accuracy	Best For
`base`	~5-7 min	Good	English, fast drafts
`large`	~30+ min	Excellent	Accented speech, final SRT

⚠️ **LESSON: Whisper outputs Urdu script for Hindi audio.** The model gets confused between Hindi and Urdu. Workaround: use Whisper for timing data, then manually write Devanagari SRT using the known script.

YouTube Uploads

⚠️ **LESSON: YouTube has a daily upload cap** (separate from API quota). Error: `uploadLimitExceeded`. Fix: wait 24h and retry. `youtube_token.json` is separate from Drive `token.json`.

---

16. 📦 Large File Handling

⚠️ **Rule #1: WhatsApp = 16MB max.** Everything larger goes through Google Drive.

Streaming from Google Drive with ffmpeg

```
ffmpeg -headers "Authorization: Bearer {access_token}" \
  -ss 00:05:00 -i "https://www.googleapis.com/drive/v3/files/
        {file_id}?alt=media" \
  -t 00:02:00 -c copy output.mp4
```

HTTP range seeking works — you can jump to any timestamp in multi-GB files without downloading the whole file.

⚠️ **LESSON: MOV files often have the moov atom at the END** of the file. Partial downloads are unreadable. Always stream from the Drive URL instead of downloading.

Audio Processing

```
# Silence detection and removal
ffmpeg -i input.mp4 -af silencedetect=noise=-30dB:d=0.5 -f null -
        2>&1 | grep silence

# Extract specific audio channel (e.g., channel 8 of 8-channel PCM)
ffmpeg -i input.wav -af "pan=mono|c0=c7" output_ch8.wav
```

---

17. 🔧 Common Issues & Fixes

Issue	Cause	Fix
	Daily upload cap	Wait 24h, retry

Issue	Cause	Fix
YouTube `uploadLimitExceeded`		
WhatsApp media not sending	File >16MB	Upload to Drive, share link
iconik API `App-ID required`	Proprietary UUID needed	Skip iconik, use Drive link
ffmpeg drawtext not working	libfreetype not compiled	Use Pillow for text overlays
Google token 401	Access token expired	`creds.refresh(Request())`
Whisper outputs Urdu for Hindi	Model language confusion	Manually write Devanagari SRT
Agent not responding	Gateway stopped	`openclaw gateway restart`
`op signin` fails	1Password app not linked	Enable app integration in 1Password settings
YouTube/Drive scope error	Wrong token file	Check: `youtube_token.json` vs `token.json`
ffmpeg SIGTERM on large files	Process killed by OS	`nohup` + Python subprocess, `timeout=7200`
`declare -A` fails in shell	macOS uses zsh, not bash	Use zsh syntax or Python
Agent sends unwanted DMs	SOUL.md boundaries not set	Add explicit "no unsolicited DMs" rule

---

18. 📖 The Prompt Engineering Bible

Operating Principles

1. **Scope first, execute second** — always confirm the plan before heavy work. "Here's what I'll do: [steps]. Proceed?"

2. **One task at a time** — complex parallel tasks cause queue confusion and context blowout.
3. **Give context, not just commands**
    - ❌ "Make a reel"
    - ✅ "Make a 16:9 comeback reel from Map3 footage, silence-trimmed, Vine Boom transitions, hero's journey narrative arc, 90 seconds max"
4. **Teach the soul, not just the skill** — a well-designed SOUL.md means the agent KNOWS when to ask, when to act, and when to stop. You shouldn't need to micromanage.
5. **Train progressively** — start simple, add complexity as trust builds.
6. **Memory is everything** — regularly flush sessions to memory. The agent only knows what's written down. If you had a great conversation but didn't save it, it never happened.
7. **The "Jobs Done" protocol** — for tasks >60 seconds, set up a completion signal. The agent should notify you when done.
8. **Model selection matters**
    - **Haiku**: quick lookups, simple Q&A
    - **Sonnet**: 90% of daily work
    - **Opus**: complex documents, deep reasoning, multi-step analysis

---

19. 💰 Cost Estimates (Monthly)

Use Case	Model	Est. Cost/ Month
Daily chatbot + reminders	Sonnet	$10–30
Content pipeline (drafts, scripts, edits)	Sonnet	$20–60
Heavy coding agent	Sonnet/Opus mix	$50–150
Full dual-agent setup (Ares + Sam)	Sonnet primary	$30–80

Use Case	Model	Est. Cost/ Month
One-off complex documents	Opus	$5–20/doc

💡 **Biggest cost driver:** context length. Long conversations with lots of tool output burn tokens fast. Flush to memory and start fresh sessions regularly.

---

20. 📋 Quick Reference Card

Gateway Commands

```
openclaw gateway start       # Start the daemon
openclaw gateway stop        # ⛔ Kill switch — stops everything
openclaw gateway restart     # Restart cleanly
openclaw gateway status      # Health check
openclaw dashboard           # Open web UI
```

Chat Commands

```
/status      — Show model, usage, cost
/reasoning   — Toggle reasoning display
/verbose     — Toggle verbose output
sonnet       — Switch to Sonnet model
haiku        — Switch to Haiku model
Opus46       — Switch to Opus 4.6
```

Memory

```
"Distill this session to memory"     — Flush current session
"What's in your memory about X?"     — Query memory
```

🔴 Emergency Kill Sequence

```
# 1. Stop gateway (immediate)
openclaw gateway stop

# 2. If gateway won't stop, kill processes
```

```
ps aux | grep openclaw | grep -v grep | awk '{print $2}' | xargs kill
        -9

# 3. Remove API key (prevents restart)
# Edit ~/.openclaw/openclaw.json → delete ANTHROPIC_API_KEY

# 4. Disable soul (agent loses identity)
mv ~/clawd/SOUL.md ~/clawd/SOUL.md.disabled
```

Useful Links

- 📖 Docs: docs.openclaw.ai
- 🛒 Skills: clawhub.com
- 💬 Community: Discord

> *"Start locked down. Peel layers slowly. The agent earns trust the same way a new employee does — one successful task at a time."* — Rushi

---

This document was generated by Sam, Rushi's content agent, running on OpenClaw. Meta enough for you? 🐾